OGUNSOLA TOLUWASE PAUL

Distributed Systems & AI Infrastructure Engineer — correctness under concurrency, retries, and partial failure | TypeScript · Node.js · Python

Oyo State, Nigeria | Open to Remote Roles & Contract Work
oluwaseyiogunsola90@gmail.com | +234 806 700 4668
GitHub | LinkedIn | Portfolio

CURRENTLY BUILDING

agent-broker — a TypeScript and Python library that answers "is this LLM call allowed to happen right now?" across multi-agent architectures: independent processes sharing budget, delegation depth, concurrency, and provider-failure state through atomic Redis Lua scripts. Ten agents each retrying twice send a provider twenty requests it never agreed to absorb; two agents checking budget milliseconds apart both pass and both spend. agent-broker makes those races impossible — one atomic admission script fuses existence, circuit, budget, and concurrency checks. Published on npm (v0.1.1); the Python port runs the same Lua scripts against the same Redis keys, so a TypeScript process and a Python process draw from one shared pool and one circuit breaker. LangGraph integration in progress (unreleased). GitHub

PROFESSIONAL SUMMARY

I build systems that stay correct when processes race, crash, and disagree. Three years shipping TypeScript, Node.js, and Python services on PostgreSQL and Redis, with a focus on the failure modes that only appear at the boundaries between processes. That means append-only financial ledgers that reject mutation even when the application layer is bypassed, idempotent workflows that collapse duplicate submissions to a single row, cross-process coordination that keeps a shared budget from being overdrawn, and multi-provider LLM infrastructure that fails over mid-stream without dropping the request. The through-line is evidence over assertion — every claim below is backed by a benchmark, a multi-process harness, or a documented experiment, not a code review.

Engineering focus: correctness under concurrency, crashes, and partial failure, with AI infrastructure (multi-provider LLM failover, cost/concurrency governance) as a growing specialty.

TECHNICAL SKILLS

Languages: TypeScript, JavaScript, Python, SQL, Lua

Backend Engineering: Node.js, Express.js, FastAPI, Flask, REST APIs, WebSockets, Socket.IO, Gunicorn, APScheduler

Databases & Data Modeling: PostgreSQL, SQLAlchemy, Supabase Migrations, Row-Level Locking, PostgreSQL RPCs, Partial Indexes, GIN Indexes, Functional Indexes

Distributed Systems & Background Processing: Redis, Atomic Lua Scripts, Redis Pipelines, Sorted Sets, Distributed Locking, Idempotency Design, Circuit Breakers, Budget Reservation and Reconciliation, Crash Recovery, Multi-Provider LLM Failover, BullMQ, RQ, Scheduled Jobs, Retry and Backoff Policies, Rate Limiting, npm Package Publishing, Library/SDK API Design, Cross-Language Protocol Design, Architecture Decision Records

Testing & Quality: Jest, Pytest, Vitest, Integration Testing, Multi-Process IPC Test Harnesses, Concurrency Testing, Fault Injection, Benchmarking, Cross-Language Testing, mypy --strict, GitHub Actions CI (path-filtered workflows)

Infrastructure, Security & Observability: Docker, Docker Compose, Git, GitHub Actions, CI/CD, Vercel, Railway, AWS, Cloudinary, Firebase Cloud Messaging (FCM), JWT, Refresh-Token Rotation, Replay Detection, RBAC, Tenant Isolation, Sentry Alerting

Frontend Exposure: React.js, React Native, Expo

EXPERIENCE

BirthdayMessaging — Backend Engineering Intern

Oct 2026 – Present

Backend for a new mobile app built on top of the company's existing Facebook birthday-messaging Chrome extension, already live on the Chrome Web Store (Manifest V3, DOM-automation based: simulates a real user composing and sending Messenger messages and timeline posts). Built the backend from scratch while a teammate built the mobile app's React Native frontend against it.

- **Backend Build (FastAPI + SQLAlchemy 2.0):** Built the mobile backend end to end on async FastAPI and SQLAlchemy 2.0 over PostgreSQL. Auth is JWT access + single-use refresh tokens per session — **reuse or expiry kills the whole session, not just the one token**, closing the "stolen token outlives detection" gap most refresh-token implementations leave open.
- **Dual-Pass Reminder Scheduler:** Designed a scheduler running every 15 minutes under a Postgres session-level advisory lock that auto-releases on crash, running two independent passes per reminder: a lead-time pass that nudges the user ahead of a contact's birthday, and a separate pass keyed to the contact's actual birthday date that sends the real message. **Both passes share one idempotency record per reminder per year, so a reminder can never double-fire even when both passes land on the same day.**
- **Infrastructure & Integrations:** Adapted the Chrome extension's own Redis-backed rate limiter to the mobile backend, **re-keyed from the extension's Facebook-user-ID scheme to IP-based** since mobile has no equivalent identity at that layer, with

automatic in-memory fallback so a Redis outage degrades limiting rather than taking auth down. Added Cloudflare R2 presigned-upload object storage scoped per user, and PayPal Orders-API payment capture with webhook-confirmed activation.

- **Testing:** Authored automated tests covering backend functionality and failure scenarios (auth, scheduler timing/idempotency, payments, uploads) — 162 passing at time of verification. **Backend feature-complete end to end and merged to main.**
- **Technical Pushback & Architecture Input:** When initially scoped, the mobile app was expected to replicate the Chrome extension's Facebook birthday-message automation directly. **Raised this as infeasible on mobile and proposed the scheduler-plus-approval-queue design actually shipped instead** (push/email auto-send, WhatsApp deep-link hand-off) after discussion with the founder and engineering manager. Also proposed Cloudflare R2 for photo storage over raw base64 and Redis for rate limiting and session caching, to make both horizontally scalable; both were adopted.
- **Architecture Contributions to the Chrome Extension:** Proposed and reviewed architecture changes for the company's existing Chrome extension, published on the Chrome Web Store, which automates sending birthday messages on Facebook by simulating real user actions in the browser — navigating to a contact's profile or Messenger thread, composing a message, and sending it.
- **Mentorship:** Mentored engineers across the team in meetings and technical discussions — **contributing architecture suggestions and reviewing approach with less experienced engineers.**

PROJECTS

agent-broker — Cross-Process Coordination Layer for Multi-Agent Architectures | GitHub Repository | npm
Creator & Maintainer | *Sep 2026 – Present*

A Redis-backed admission gate for multi-process agent and LLM-calling systems, shipped as an in-process library (no broker service to deploy) with TypeScript (published on npm) and Python implementations coordinating through one shared Redis protocol.

Impact snapshot: Published on npm (v0.1.1) · TypeScript + Python implementations sharing one Redis protocol, verified by cross-language tests · 20 forked OS processes racing 10 budget slots → exactly 10 admitted, zero deviation · 20 racing 3 concurrency slots → exactly 3 · 1 atomic 7-key admission script · 7 typed denial reasons · 26 ADRs (including rejected alternatives) · admission ~0.25ms median / ~4–5ms p99 (single-core local Redis, sequential)

- **Atomic Admission Engine:** Fused agent-existence, circuit-breaker, budget, and concurrency checks into a single 7-key Redis Lua script (EVALSHA with EVAL fallback), eliminating the check-then-act races that let uncoordinated processes overspend a shared pool, exceed concurrency limits, or stampede a failing provider. Denials are typed across 7 reasons — 4 from the admission script (unknown_agent, budget_exceeded, concurrency_exceeded, circuit_open) and 3 from queue/degraded mode (aborted, queue_timeout, redis_unavailable) — so callers can distinguish "wait and retry" from "this will never succeed." Verified under real OS-process contention: **20 forked processes racing 10 budget slots admit exactly 10 every run; 20 racing 3 concurrency slots admit exactly 3**, with every remaining denial reported as concurrency_exceeded rather than a mixed reason set. Admission Engine Code
- **Caller-Unforgeable Delegation Trees:** Enforced broker-derived agent hierarchies via an atomic REGISTER_CHILD Lua script that checks parent existence, computes depth, and derives rootId and budgetKey inside Redis in one round trip — **closing a time-of-check-to-time-of-use race where a parent could expire mid-delegation and spawn orphan children**. A sub-agent's depth is never caller-supplied, so a child can't claim a shallower depth than its parent tree implies (ADR-0008).
- **Two-Phase Reconciliation & Crash Recovery:** Built a requestPermission / reportOutcome API that reserves estimated cost pre-call and reconciles actual usage post-call through one shared resolution script, so an explicit report and the lazy sweep of an abandoned reservation resolve through the same idempotent path rather than two cleanup mechanisms that quietly drift apart. **Verified with a genuinely killed process (real SIGTERM, not a simulation): the next caller's sweep refunds the budget and releases the concurrency slot exactly once**, and a late report racing the sweep is rejected as already_resolved rather than double-refunding. Cleanup deliberately never feeds the circuit breaker — an abandoned reservation proves the caller crashed, not that the provider is failing (ADR-0001).
- **Sliding-Window Circuit Breaker with Probabilistic Probe Recovery:** Built a Redis sorted-set failure window with soft backpressure (a retryAfter hint) and a hard threshold that opens the circuit for every process sharing the target, not just the process that saw the failures. Recovery admits a probabilistic single probe — Math.random() compared to probeRate inside the same atomic script as the failure count and budget checks — so concurrent callers can't all read the same pre-increment state and flood a recovering provider with simultaneous probes; a single successful probe closes the circuit for all waiting processes. **Verified cross-process: a process with zero local failure history is denied purely by other processes' failures.**
- **Cross-Language Protocol (TypeScript + Python):** Shipped a feature-complete async Python implementation (mypy --strict, CI across Python 3.10–3.13) that runs byte-identical Lua scripts against the same Redis keys as the TypeScript package, so a TypeScript process and a Python process register agents, request permission, and resolve reservations against one shared budget pool, one concurrency limit, and one circuit breaker — verified cross-language: one language's failures open a circuit the other observes, and one language's sweep recovers the other's abandoned reservation. **Two drift guards fail CI if the Lua scripts or the public exports diverge, and a language-neutral fixture is replayed through both workers and must produce identical results.**
- **Published Library & API Design:** Released to npm with a packed-tarball consumer smoke test and an ioredis v5/v6 peer range verified in CI. Designed the surface for adoption: bring-your-own Redis client (the library never owns a connection), immutable per-instance config, opt-in queue mode with backoff polling and AbortSignal cancellation returning immediately on denials

waiting can never fix (unknown_agent), configurable onRedisUnavailable: 'deny' | 'allow' so an outage never silently drops the coordination guarantees, and a documented trust model stating what the library does not protect against. **Documented limitation: not designed for sub-10ms workloads — admission costs one Redis round trip by design.**

Kith — Financial Coordination System | GitHub Repository | Live Demo
Creator & Backend Engineer | *Mar 2026 – Sep 2026*

Family financial coordination platform with an immutable ledger, automated recurring pools, and a 9-queue background job system, documented in 20 Architecture Decision Records.

Impact snapshot: 23-table schema · 9 PostgreSQL RPCs, 7 for multi-step financial workflows · 9-queue BullMQ · 124-operation REST API · zero invariant violations under 30 concurrent duplicate submissions across 9 runs · 1,000+ backend test cases + 83 frontend tests · membership cache p50 ~14x / p95 ~5x faster (1.2–1.7ms cached vs 8.1–8.8ms uncached)

- **Database-Enforced Ledger Immutability:** Designed an append-only ledger where confirmed amounts are immutable and corrections are linked rows, enforced at the database layer by a BEFORE UPDATE/BEFORE DELETE trigger that rejects direct SQL mutation of confirmed entries. **Verified by a raw UPDATE attempt (rejected, P0001)** while sanctioned status transitions still pass; an audit_log_append_only trigger extends the same protection to the audit trail (30 action types), immune to mutation even from the service-role key.
- **Concurrency-Safe Financial Workflows:** Moved 7 multi-step write workflows into atomic PostgreSQL RPCs. **Held the one-current-target invariant under 5/25/50 concurrent requests across 5 runs with zero violations**, enforced by a partial unique index on a 23-table schema (65 foreign keys, 27 CHECK constraints, 49 indexes). The target-setting RPC retries on unique_violation (bounded, 5 attempts) so concurrent callers serialize through the lock instead of surfacing a raw DB error.
- **Idempotent Ledger Writes Under Concurrency:** Verified via 9 concurrent-request runs (concurrency 5/15/30) that duplicate submissions sharing one idempotency key resolve to exactly one ledger row, via explicit PostgreSQL 23505 unique-violation handling. **100% pass rate, zero errors surfaced to callers**, losing requests receiving a clean idempotent-replay response. A separate 10-minute duplicate-detection heuristic (pg_advisory_xact_lock-guarded) catches the different failure mode of a human double-submitting by mistake, with an explicit force=true override path.
- **Idempotent Unattended Automation & Transactional Outbox:** Nine BullMQ queues, seven cron jobs, per-queue concurrency tuning (1–10, with a 50 jobs/sec limiter on notification delivery). Cycle generation and lifecycle automation are idempotent by database constraint — unique indexes keyed on cycle identity and carry-forward source — so a re-run can never double-apply. **Push/email deliveries use a transactional outbox pattern: a failed enqueue is marked failed, never silently dropped**, then a 5-minute recovery sweep re-enqueues stranded deliveries with a capped retry budget.
- **Tenant Isolation, Tiered Rate Limiting & Security Hardening:** Enforced tenant isolation and RBAC through a JWT → membership → role middleware chain backed by Redis caches with 30–60s TTLs and explicit invalidation on role/status changes, alongside 6 Redis-backed rate limiters protecting pre-auth and post-auth tiers. **Measured membership-lookup speedup at ~14x p50 / ~5x p95** (4 runs × 300 calls, local benchmark) versus the uncached two-query path (1.2–1.7ms cached, 8.1–8.8ms uncached). Security hardening: magic-byte upload verification, OAuth redirect allowlist, and a dedicated limiter against token enumeration on public lookup endpoints.
- **Testing, CI/CD & Observability:** Authored automated test suites across 3 Jest projects (unit, integration, rate-limit) — **1,000+ backend test cases plus 83 frontend tests**, with integration and rate-limit suites run against a 3-container PostgreSQL, PostgREST, and Redis stack — all four suites (unit, integration, rate-limit, frontend) now run in CI on every push; 90%+ unit coverage on validators and middleware, 80%+ on core services, from hand-verified per-project runs.

StudyHub — AI & Real-Time Infrastructure | GitHub Repository | Live Demo
Creator & Backend Engineer | *Jan 2025 – Present*

Peer academic collaboration platform featuring a multi-provider LLM failover engine, Redis-coordinated real-time services, and distributed background-job coordination.

Impact snapshot: 52-model ORM · 43 indexes · 6-provider/14-model LLM failover · 29 Socket.IO handlers · 5 Redis key families · 0 mutual-exclusion violations across 90 trials at 25 concurrent processes · 37/37 fault-classifier boundary cases · 1,458 passing unit + integration tests

- **Relational Schema & Race-Closing Indexes:** Engineered a 52-model SQLAlchemy ORM on PostgreSQL with 43 explicit indexes, including partial, GIN, and functional unique indexes that **close bidirectional-duplicate race conditions at the database level**, plus composite and partial indexes matched to actual unread-count query shapes.
- **Real-Time Presence & Message Dispatch:** Built 29 inbound Socket.IO handlers with multi-device presence tracking across 3 Redis key types (120s TTL, 45s refresh sweep). **Self-healing on read** means a stale entry is pruned by the next lookup rather than needing a separate sweep; cut hot-path dispatch cost with Redis pipelines (MGET/SADD) and ThreadPoolExecutor offloading. Presence Code · WebSocket Code
- **Multi-Provider AI Resilience Engine:** Architected a framework-agnostic LLM routing layer across 6 vendors and 14 models, with Redis-cached model discovery, stripping of leaked reasoning-tag output (<think>/<reasoning>) from model responses, and mid-stream SSE failover that moves to the next provider without dropping the request. **Built a fully-functional non-AI fallback for one feature, so the AI layer is never a single point of failure for a problem with a reasonable deterministic answer.** AI Provider Code

- **Failure Taxonomy & Circuit Breaking:** Classified upstream errors into 4 fault classes (KEY_FAULT, PROVIDER_TRANSIENT, BAD_MODEL, NON_RETRYABLE) **from HTTP status and structured error bodies rather than string matching**, so a provider-wide outage advances to the next provider without cooling a healthy API key, and a "model not found" response evicts only that model from the shared cache. Enforced 1-hour per-key cooldowns and a 30-minute provider-type blacklist after 3 failures in 5 minutes, gated behind a fail-open Redis kill switch (AI_PROVIDER_REDIS_STATE_ENABLED). Validated against a 37-case boundary harness, 37/37 passing.
- **Distributed Locking & Deliberate Failure Modes:** Built 5 lock-guarded scheduled jobs (atomic SET NX EX acquisition, Lua compare-and-delete release) that deliberately fail closed, because a duplicate leaderboard snapshot write is worse than a skipped tick, while rate limiting, AI provider state, and error tracking deliberately fail open, because availability beats correctness for each of those. Added 2 durable RQ queues with per-job retry and backoff. **Verified mutual exclusion across 90 trials at up to 25 concurrent OS processes with zero violations, against up to 4,750 violations in a naive control run through the identical harness**; separately confirmed fail-closed behavior across 3 Redis-outage simulations (30/30 trials).
- **Concurrency Controls & Secure Token Rotation:** Closed capacity and duplicate-insert races with 3 SELECT FOR UPDATE sites, 2 transaction savepoints, and database-level unique indexes. **Demonstrated zero lost updates across 15 trials (up to 100 simultaneous OS processes) on the Redis INCRBY-backed unread counter, where a naive read-modify-write control lost a median of 63% of updates at the same concurrency level.** Implemented SHA-256-hashed refresh-token rotation with family_id replay detection and a 10-second grace window that separates a legitimate multi-tab race from a stolen-token replay. Hardened further with CSRF double-submit enforcement, a Sentry PII scrubber, and Pillow verify-and-re-encode image uploads that reject SVG-as-.jpg polyglot payloads.
- **Testing, CI & Enforced Architecture:** 1,458 passing unit and integration tests run on every push across two GitHub Actions workflows against SQLite and fakeredis, with fakeredis plus lupa executing the real Lua lock-release script rather than mocking it. A scheduled drift-reconciliation job auto-corrects cosmetic counters but never auto-corrects capacity-gating Thread.member_count, which is alert-only since a wrong "fix" could silently over- or under-admit members. scripts/check_layering.py **statically parses every service file's AST and fails the build if business logic imports Flask request objects or reaches into routes/**, which is what lets HTTP routes, WebSocket handlers, and background jobs safely call the same service functions.

TECHNICAL WRITING

- **Designing agent-broker: Collision-Safe Redis Keys, Budget Integrity, and Broker-Derived Delegation Identities** *(companion to agent-broker)*
  Explored collision-safe Redis key design, atomic budget reservation and reconciliation, missing-resource handling, and broker-controlled delegation identities. Read it
- **Reengineering Family Finance: Building Financial Integrity Into Kith** *(companion to Kith)*
  Discussed financial data integrity through database-enforced ledger immutability, idempotent operations, atomic PostgreSQL transactions, and reliable background job processing. Read it
- **Not All AI Provider Failures Are the Same: Building a Fault-Aware LLM Fallback System** *(companion to StudyHub)*
  Examined LLM provider failure classification, differentiated retry and failover strategies, provider health management, and Redis-backed state coordination. Read it

EDUCATION

Federal University of Technology, Akure (FUTA) — Ondo State, Nigeria
Bachelor of Science (B.Sc.) in Computer Science — Expected Graduation: 2028